

Mastering State (useState) and Side Effects (useEffect)

State & Basic Hooks (useState)

1. What is State?

In React, State represents a component's short-term memory. It is the internal data that changes over time based on user interactions (e.g., button clicks, form inputs, toggles).

- **The Problem with Normal Variables:** Standard JavaScript variables (e.g., `let x = 0`) reset every time a component function re-runs (re-renders). They do not persist across renders, and changing them does not signal React to update the screen.
- **The State Solution:** React stores state outside the component function in its internal memory bank. This keeps state safe across re-renders and automatically triggers a screen redraw whenever the state changes.

2. The Problem Code: Why Normal Variables Fail

Consider this common mistake where a developer attempts to track user input or button clicks using a standard variable:

```
// ❌ BROKEN: Using a normal variable
export function BrokenCounter() {
  let count = 0; // Resets back to 0 on every render!

  const handleClick = () => {
    count = count + 1; // 1. Value changes in memory internally...
    console.log("Count is now:", count);
    // 2. BUT React doesn't know, so the UI on screen never updates!
  };

  return (
    <button onClick={handleClick}>
      Clicked {count} times (I don't work!)
    </button>
  );
}
```

Using useState

By replacing the standard variable with React's `useState` hook, the variable persists across renders, and modifying it notifies React to re-render the component:

```
// FIXED: Using useState
import { useState } from 'react';

export function WorkingCounter() {
  // useState preserves memory across renders and alerts React to update UI
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times (I work!)
    </button>
  );
}
```

Syntax & Anatomy Breakdown

The useState hook uses JavaScript array destructuring:

```
const [count, setCount] = useState(0);
```

- 1. State Variable (count):** Holds the current value stored in memory.
- 2. Setter Function (setCount):** A function that updates the state variable and tells React to re-render.
- 3. Initial Value (0):** The starting value provided to state on the initial page load.

Core Rules of State

- **Never Mutate State Directly:** Never write `count = count + 1`. Always call `setCount(count + 1)`. Direct mutation circumvents React's reactivity engine.
- **State Updates are Asynchronous:** Calling `setCount()` schedules a re-render; it does not instantly change the count variable within the current function execution frame.

Side Effects (useEffect)

What is a Side Effect?

A component's primary responsibility is calculating and rendering HTML based on props and state. A side effect is any operation that reaches outside the scope of the component function to perform external tasks:

- Fetching data from a backend server/API

- Setting up timers or intervals (e.g., setInterval)
- Manually modifying the browser DOM (e.g., document.title = ...)

2The Problem Code: Uncontrolled Side Effects & Infinite Loops

If an asynchronous operation or side effect (like fetching data and updating state) is placed directly inside the body of a component, it causes a catastrophic infinite render loop:

```
import { useState } from 'react';

// BROKEN: Fetching data in the main function body
export function InfiniteLoopComponent() {
  const [data, setData] = useState(null);

  // DANGER:
  // 1. Component renders
  // 2. fetch() runs
  // 3. .then() calls setData()
  // 4. setData() triggers a re-render
  // 5. Component re-renders -> fetch runs again -> INFINITE LOOP!
  fetch('https://api.example.com/data')
    .then((res) => res.json())
    .then((result) => setData(result));

  return <div>Data: {JSON.stringify(data)}</div>;
}
```

The Working Code: Wrapping Operations in useEffect

The useEffect hook allows you to decouple side effects from the immediate rendering flow, executing code only at controlled moments:

```
import { useState, useEffect } from 'react';

// ☒ FIXED: Wrapping the fetch inside useEffect with a dependency array
export function SafeFetchComponent() {
  const [data, setData] = useState(null);

  useEffect(() => {
    // React runs this ONLY once after the initial render because of []
    fetch('https://api.example.com/data')
      .then((res) => res.json())
      .then((result) => setData(result));
  }, []); // Empty dependency array = "Run once on mount"
```

```
    return <div>Data: {JSON.stringify(data)}</div>;  
  }
```

4. Dependency Array Cheat Sheet

The second argument of `useEffect` determines when the effect callback function runs:

Dependency Array	Execution Timing	Common Use Case
No Array <code>useEffect(fn)</code>	Runs after EVERY render.	Rarely used; general logging/analytics.
Empty Array <code>useEffect(fn, [])</code>	Runs ONCE on initial load (mount).	Initial data fetching on page load.
With Dependencies <code>useEffect(fn, [id])</code>	Runs on mount AND when listed dependencies change.	Re-fetching data when a prop or dropdown selection changes.

Effect Cleanup

When side effects open ongoing connections (e.g., subscriptions, event listeners, or timers), they must be cleaned up to prevent memory leaks when the component unmounts:

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Tick...");  
  }, 1000);  
  
  // Return a cleanup function: React runs this before unmounting/re-running effect  
  return () => {  
    clearInterval(timer);  
  };  
}, []);
```